

Optimizations Implemented

We have chosen to implement 5 optimisations:

1. **Constant Folding** - When we encounter an assignment to a binary expression with operation of +, -, *, /, where both operands are constant integers, we just immediately calculate the result and use it to replace the binary operation.

How we implemented this:

We implement this in [IROptimizer.py](#). It scans through our IR and looks for these situations. Whenever we find them, we simply replace the expression.

2. **Strength Reduction** - When we encounter expensive operations that could be simplified, we convert them to a simpler form. We support the simplification of 4 such operations:
 - Converting multiplication by a power of 2 into shift left by $\log_2$ of that amount
 - Converting $\text{var2} * 1$ into var2
 - Converting $\text{var2} * 0$ into 0
 - Converting $\text{var2} + 0$ into var2

How we implemented this:

We implement this in [IROptimizer.py](#). It scans through our IR and looks for these patterns. Whenever we find them, we simply replace the operation.

3. **Dead Code Elimination** - When we encounter variables that are never used and unreachable code. Examples of unreachable code include code after unconditional jumps (as the jump will always occur), or code blocks within always-false conditions.

How we implemented this:

We implement this in [IROptimizer.py](#). For variables, it removes assignments to temporary variables if they are never used later. For unreachable code elimination, the optimizer performs a reachability analysis over the IR. It begins at the first instruction and simulates execution by exploring all possible control flow paths. Specifically, it follows unconditional jumps ([goto](#)), conditional branches ([if ... goto](#)), function calls ([FuncCall](#)), and normal fall-through to the next instruction. As it traverses these paths, it marks each visited line as reachable. The optimizer also evaluates constant conditions (e.g., [if True](#), [if 3 < 5](#)) to determine which branch will actually be taken, allowing it to avoid exploring impossible paths. After this exploration, any instruction that was never marked as reachable is considered dead code and removed from the IR. This ensures that only code that can actually execute remains in the optimized output.

4. **Conversion to capacity based lists** - We redesigned our list implementation to reduce unnecessary memory allocations and better align with how Python handles dynamic arrays. Instead of reallocating memory on every operation, the list now maintains a capacity separate from its current size. New elements are added without resizing until

this capacity is reached, and only then is the underlying storage expanded by a growth factor. This approach improves efficiency by minimizing the number of reallocations.

How we implemented this:

We implement this by simply reworking the structs that we append to the start of files. Now instead of reallocating on every change in list size, we allocate based on capacity (which grows and shrinks by a multiplicative factor of 2), and reallocate only when capacity is exceeded, which should be exponentially less reallocations in most circumstances.

This can be seen by comparing the original struct file [python_list_struct.c](#) with the new optimised version [python_list_struct_optimized.c](#).

5. **Removal of un-needed boilerplate code** - If a program does not make use of lists, we remove all list structs.

How we implemented this:

We implement this by scanning the IR for an instance of "[]", the empty list, which must be present at some point if a list is used. If no such "[]" exists, then we avoid adding the list structs. This is implemented directly in the [CGen.py](#).

[IROptimizer.py](#) is run after the IR generation code, and before the C generation code.

How to run our code

As before, use [run.sh](#) to compile all of our code and execute it, automatically comparing the C and Python versions.

This time however, we also automatically include the optimisations. To opt out of the optimisations, simply include the flags "-noOpt1 -noOpt2 ...", where noOptN removes the optimisation N, using the same numbering as above.

In order to facilitate examining our optimisations, we have added a few new examples (examples 1 to 7 remain the same as last sprint).

Example 8 is meant for testing optimisation 4. It simply appends to an empty list 10 000 times. The goal for this was to see how many fewer allocations this optimization causes, which we discuss below.

Example 9 is meant for testing optimisation 3. It is a program that has paths that are not taken. We simply remove the paths. To see the difference just compare the resulting code in `target_output/example9.c` with and without the -noOpt3 flag.

Example 10 is meant for testing optimisations 1 and 2. It is just a program with a constant expression and some reducible operations that are simplified. To see the difference just

compare the resulting code in target_output/example9.c with and without the -noOpt1 / -noOpt2 flags.

To test optimisation 5, simply compare the file sizes with "ls -l ./target_output/" after doing ./run.sh with and without -noOpt5.

Lastly, to test that we continue to gracefully catch errors (like we did in the last sprint), you can run ./run.sh with the flag -error. The examples we show here are the same as last sprint.

Optimization Discussion

To test the improvement for the optimization, we created a basic program that iterates 100 000 times, adding each number from 0 to 99 999 into a basic list. Then we recorded the amount of times that a malloc/realloc statement was encountered. We got the following results:

Amount of Memory Allocations Before Optimisation	Amount of Memory Allocations After Optimisation
100 000	14

To test the improvement for the optimization, we compare the file sizes of our example code with and without the optimization. We get the following results:

	Number of Bytes Before Optimisation	Number of Bytes After Optimisation
Example 1	6539	643
Example 2	6626	730
Example 3	6163	267
Example 4	7985	7756
Example 5	6275	379
Example 6	6263	367
Example 7	6465	6395
Example 8	6237	6167

Note that the files that do have lists are still smaller because of some optimisations in the list struct itself for Sprint 4 (ex. using realloc instead of manual copying). This accounts for the approximately 200 byte decrease in file size for the examples that do need the list header. With that in mind, we see that this optimization saves approximately 5700 bytes per file. For our purposes, namely small examples, this is up to a 17 times decrease in file size. However we acknowledge for more significant applications, the size improvement is less noticeable.